R Code Examples from WDDA Lecture 04 (Chapter 4)

WDDA – Data Management and Data Analysis

Prof. Dr. Ulrich Matter

2026-04-29

Contents

1 Introduction

This document collects all R code examples from the slides of WDDA FS2026 Lecture 04 (Chapter 4). The focus is on inferential statistics, in particular sampling variation, standard errors, bootstrap methods, and confidence intervals.

Inferential statistics is the area of statistics concerned with drawing conclusions about a population on the basis of samples. In contrast to descriptive statistics, which "merely" describes the data at hand, inferential statistics aims to make statements about the entire population.

Central concepts in this area are:

1. **Sampling variation**: How much do statistics (e.g. means) vary across different samples?
2. **Standard error**: A measure of the precision of a sample statistic
3. **Bootstrap methods**: Resampling techniques for estimating standard errors and confidence intervals
4. **Confidence intervals**: Ranges that contain the true population parameter with a given probability

2 Preparation

```r
# Load packages
library(mosaic)    # e.g. for do(), resample(), dotPlot()
library(plotrix)   # for plotCI()
library(readxl)    # for reading Excel files

# Import data
Footballer <- read_excel("data/WDDA_04.xlsx", sheet = "Footballer")
BFH <- read_excel("data/WDDA_04.xlsx", sheet = "BFH")
ExerciseHours <- read_excel("data/WDDA_04.xlsx", sheet = "ExerciseHours")
Mustangs <- read_excel("data/WDDA_04.xlsx", sheet = "Mustangs")

# Inspect the structure of the data
str(Footballer)
```

```
## tibble [581 x 2] (S3: tbl_df/tbl/data.frame)
##  $ Player      : chr [1:581] "Gareth Bale" "David De Gea" "Kevin De Bruyne" "Raheem Sterling" ...
##  $ WeeklySalary: num [1:581] 600000 375000 320833 300000 290000 ...
```

R explanation:

- The function `library()` loads an installed package into the current R session.
- `read_excel()` from the `readxl` package can be used to read Excel files. The `sheet` parameter specifies which worksheet to use.
- The function `str()` displays the structure of an object, including data types and dimensions.

Note on data paths: In R Markdown files, relative paths are interpreted from the location of the file, hence the path `"../../data/WDDA_04.xlsx"`. In regular R scripts the path would be specified relative to the working directory.

3 Notation: Population vs. Sample

In statistics, we distinguish between population parameters and estimates obtained from samples. These are denoted with different symbols:

- **Population parameters**: e.g. mean , standard deviation , variance ², proportion p
- **Sample estimates**: e.g. mean $\bar{x}$, standard deviation s, variance s^2, proportion $\hat{p}$

```r
# Extract the weekly salaries
WeeklySalary <- Footballer$WeeklySalary

# Compute the population mean (here: all footballers in the file)
```

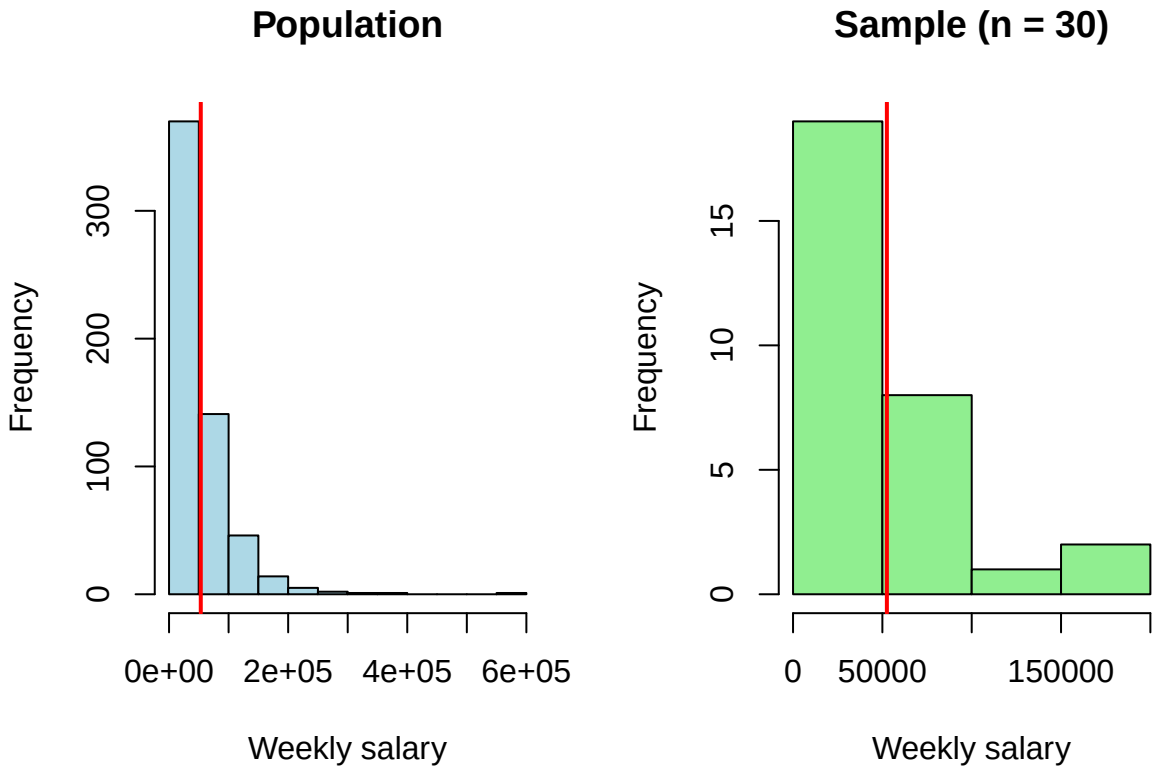
```r
mu <- mean(WeeklySalary)

# Draw a sample and compute the sample mean
set.seed(123)  # For reproducibility
stichp1 <- sample(WeeklySalary, size = 30)
xbar <- mean(stichp1)

# Print the results
cat("Population mean (μ):", round(mu, 2), "\n")
```

```
## Population mean (μ): 53280.81
```

```r
cat("Sample mean (x̄) (n = 30):", round(xbar, 2), "\n")
```

```
## Sample mean (x̄) (n = 30): 52519.8
```

```r
# Visualisation
par(mfrow = c(1, 2))
hist(WeeklySalary, main = "Population", xlab = "Weekly salary", col = "lightblue")
abline(v = mu, col = "red", lwd = 2)
hist(stichp1, main = "Sample (n = 30)", xlab = "Weekly salary", col = "lightgreen")
abline(v = xbar, col = "red", lwd = 2)
```




```r
par(mfrow = c(1, 1))
```

Statistical explanation:

- The **population mean** is the true mean of all elements in the population.
- The **sample mean** $\bar{x}$ is an estimate of , based on a random subset of the population.
- In practice we often do not know and have to estimate it using $\bar{x}$.
- Different samples yield different values of $\bar{x}$, which is referred to as **sampling variability**.

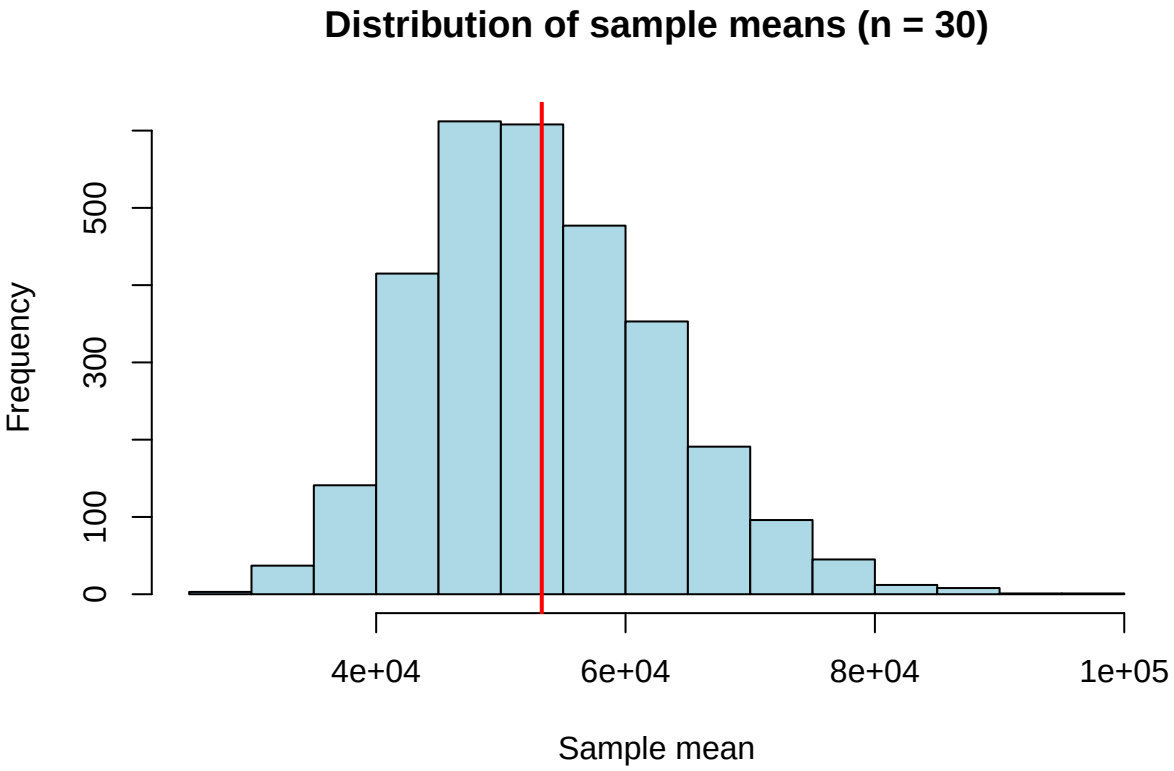
R explanation:

- The function `sample()` draws a random sample from a vector. The parameter `size` specifies the sample size.
- The function `set.seed()` sets the seed for the random number generator, ensuring reproducibility of the results.
- The function `abline()` with the parameter `v` adds a vertical line to a plot.

4 Simulating Sampling Variation

To understand the variation between different samples, we can draw many samples and look at the distribution of the sample means.

```
# Draw 3000 samples (n = 30) and compute the mean for each
set.seed(456)
m.stichps <- do(3000) * mean(sample(WeeklySalary, size = 30))

# Visualise the distribution of the sample means
hist(m.stichps$mean,
     main = "Distribution of sample means (n = 30)",
     xlab = "Sample mean",
     col = "lightblue")
abline(v = mu, col = "red", lwd = 2)
```



```r
# Alternative visualisation with dotPlot
dotPlot(m.stichps$mean,
        nint = 200,
        cex = 0.5,
        pch = 16,
        ylim = c(0, 100),
        main = "Distribution of sample means (n = 30)",
        xlab = "Sample mean")
abline(v = mu, col = "red", lwd = 2)
```

Statistical explanation:

- The distribution of the sample means is called the **sampling distribution**.
- The **central limit theorem** states that the sampling distribution of the mean is approximately normally distributed when the sample size is sufficiently large, regardless of the shape of the population distribution.
- The mean of the sampling distribution equals the population mean (unbiasedness).
- The standard deviation of the sampling distribution is called the **standard error**.

R explanation:

- The function `do()` from the `mosaic` package repeats an expression multiple times.
- The operator `*` in `do(3000) * mean(...)` indicates that the expression should be repeated 3000 times.
- The function `dotPlot()` creates a dot plot in which each point corresponds to a sample mean.

5 Standard Error of Sample Means

The standard error is a measure of the precision of a sample statistic. It indicates how much the sample statistic (e.g. the mean) varies between different samples.

```r
# Compute the standard error from the simulation
se_simuliert <- sd(m.stichps$mean)
cat("Simulated standard error (se):", round(se_simuliert, 2), "\n")
```

```
## Simulated standard error (se): 9662.53
```

```r
# Theoretical standard error from the formula se =  /√n
sigma <- sd(WeeklySalary)  # Population standard deviation
n <- 30                    # Sample size
se_theoretisch <- sigma / sqrt(n)
cat("Theoretical standard error (se):", round(se_theoretisch, 2), "\n")
```

```
## Theoretical standard error (se): 9931.83
```

```r
# Compare the variation of individual values with the variation of the means
cat("Standard deviation of the population ( ):", round(sigma, 2), "\n")
```

```
## Standard deviation of the population ( ): 54398.89
```

```r
cat("Standard error of the mean (se):", round(se_simuliert, 2), "\n")
```

```
## Standard error of the mean (se): 9662.53
```

```
cat("Ratio  /se:", round(sigma/se_simuliert, 2),
    "(approximately equals √n =", round(sqrt(n), 2), ")\n")
```

```
## Ratio  /se: 5.63 (approximately equals √n = 5.48 )
```

Statistical explanation:

- The **standard error (SE)** is the standard deviation of the sampling distribution of a statistic.
- For the mean: SE = /√n, where is the population standard deviation and n is the sample size.
- The standard error decreases as the sample size grows (proportional to 1/√n).
- The standard error is an important measure of the precision of an estimate: the smaller the standard error, the more precise the estimate.
- In practice, is often unknown and the standard error has to be estimated by s/√n, where s is the sample standard deviation.

R explanation:

- The function `sd()` computes the standard deviation of a vector.
- The function `sqrt()` computes the square root of a number.

6 Effect of Sample Size

The sample size has a decisive influence on the precision of estimates. As the sample size grows, the variation of the sample statistics becomes smaller.

```
# Draw samples of different sizes and compute the mean for each
set.seed(789)
m.30  <- do(3000) * mean(sample(WeeklySalary, size = 30))
m.100 <- do(3000) * mean(sample(WeeklySalary, size = 100))
m.500 <- do(3000) * mean(sample(WeeklySalary, size = 500))

# Compute the standard errors
se_30 <- sd(m.30$mean)
se_100 <- sd(m.100$mean)
se_500 <- sd(m.500$mean)

# Print the standard errors
cat("Standard error (n = 30):", round(se_30, 2), "\n")
```

```
## Standard error (n = 30): 9602.82
```

```
cat("Standard error (n = 100):", round(se_100, 2), "\n")
```

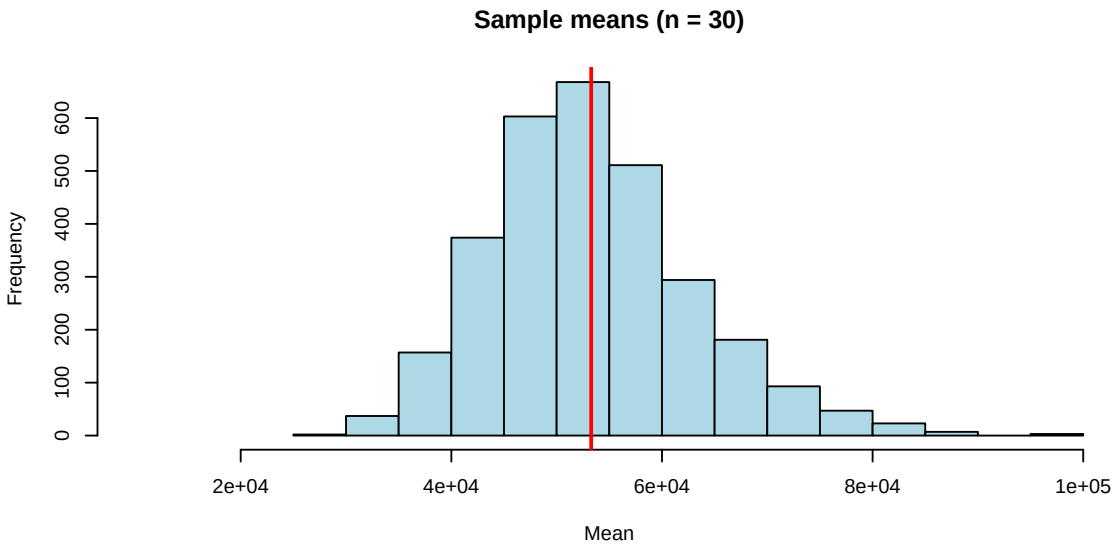
```
## Standard error (n = 100): 4940.48
```

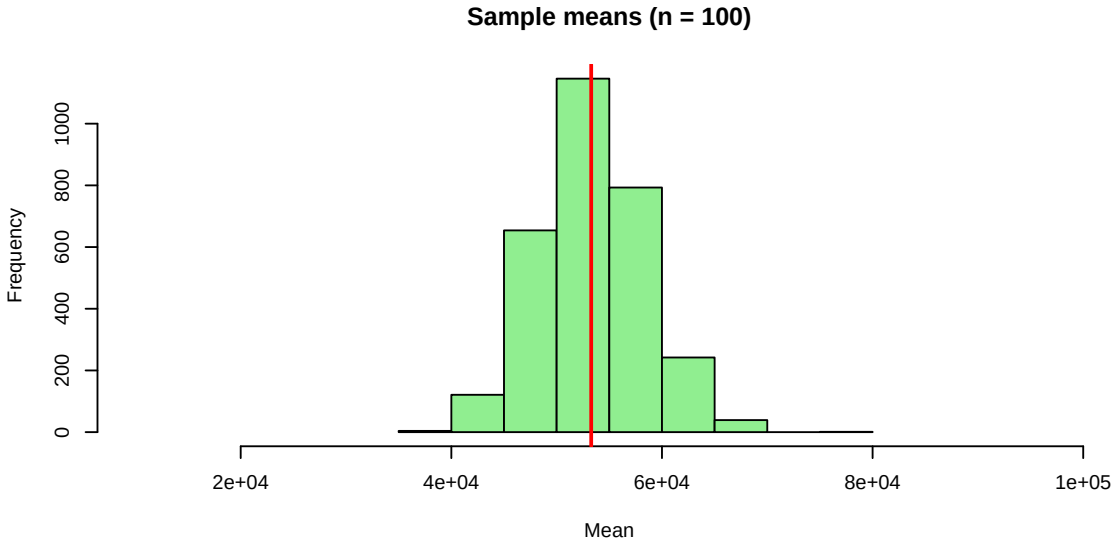
```
cat("Standard error (n = 500):", round(se_500, 2), "\n")
```

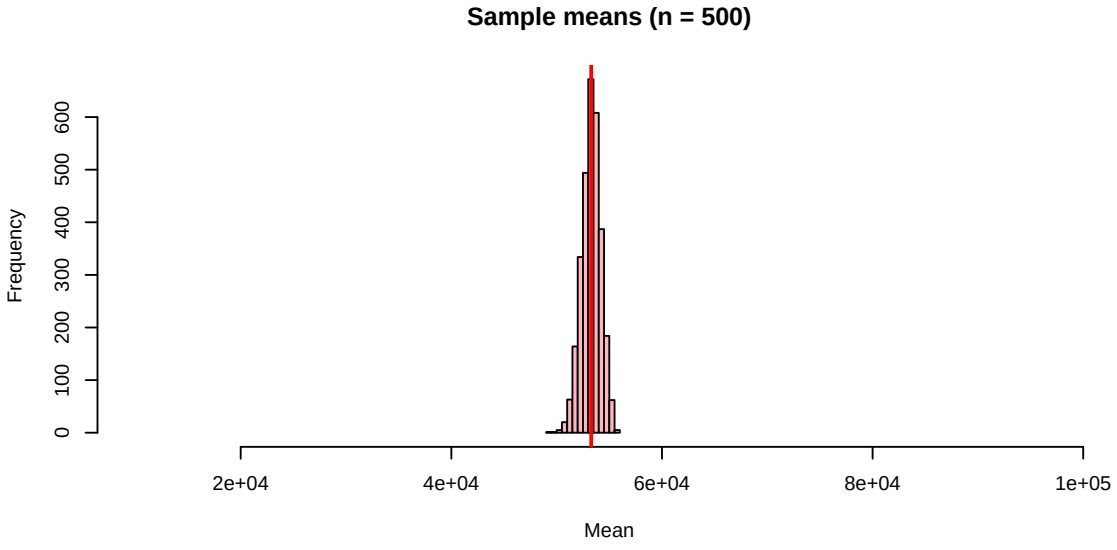
```
## Standard error (n = 500): 903.14
```

```
# Visualise the distributions
par(mfrow = c(3, 1))
hist(m.30$mean, xlim = c(10000, 100000),
     main = "Sample means (n = 30)",
```

```
     xlab = "Mean",
     col = "lightblue")
abline(v = mu, col = "red", lwd = 2)
hist(m.100$mean, xlim = c(10000, 100000),
     main = "Sample means (n = 100)",
     xlab = "Mean",
     col = "lightgreen")
abline(v = mu, col = "red", lwd = 2)
hist(m.500$mean, xlim = c(10000, 100000),
     main = "Sample means (n = 500)",
     xlab = "Mean",
     col = "lightpink")
abline(v = mu, col = "red", lwd = 2)
```


Sample means (n = 30)
Frequency
0
100
200
300
400
500
600
2e+04
4e+04
6e+04
8e+04
1e+05
Mean


Sample means (n = 100)
Frequency
0
200
400
600
800
1000
2e+04
4e+04
6e+04
8e+04
1e+05
Mean


Sample means (n = 500)
Frequency
0
100
200
300
400
500
600
2e+04
4e+04
6e+04
8e+04
1e+05
Mean

```r
par(mfrow = c(1, 1))
```

```r
# Compare theoretical and simulated standard errors
se_theo_30 <- sigma / sqrt(30)
se_theo_100 <- sigma / sqrt(100)
se_theo_500 <- sigma / sqrt(500)

cat("\nComparison of theoretical and simulated standard errors:\n")
```

```
##
## Comparison of theoretical and simulated standard errors:
```

```r
cat("n = 30:  Theoretical:", round(se_theo_30, 2),
    "Simulated:", round(se_30, 2), "\n")
```

```
## n = 30:  Theoretical: 9931.83 Simulated: 9602.82
```

```r
cat("n = 100: Theoretical:", round(se_theo_100, 2),
    "Simulated:", round(se_100, 2), "\n")
```

```
## n = 100: Theoretical: 5439.89 Simulated: 4940.48
```

```r
cat("n = 500: Theoretical:", round(se_theo_500, 2),
    "Simulated:", round(se_500, 2), "\n")
```

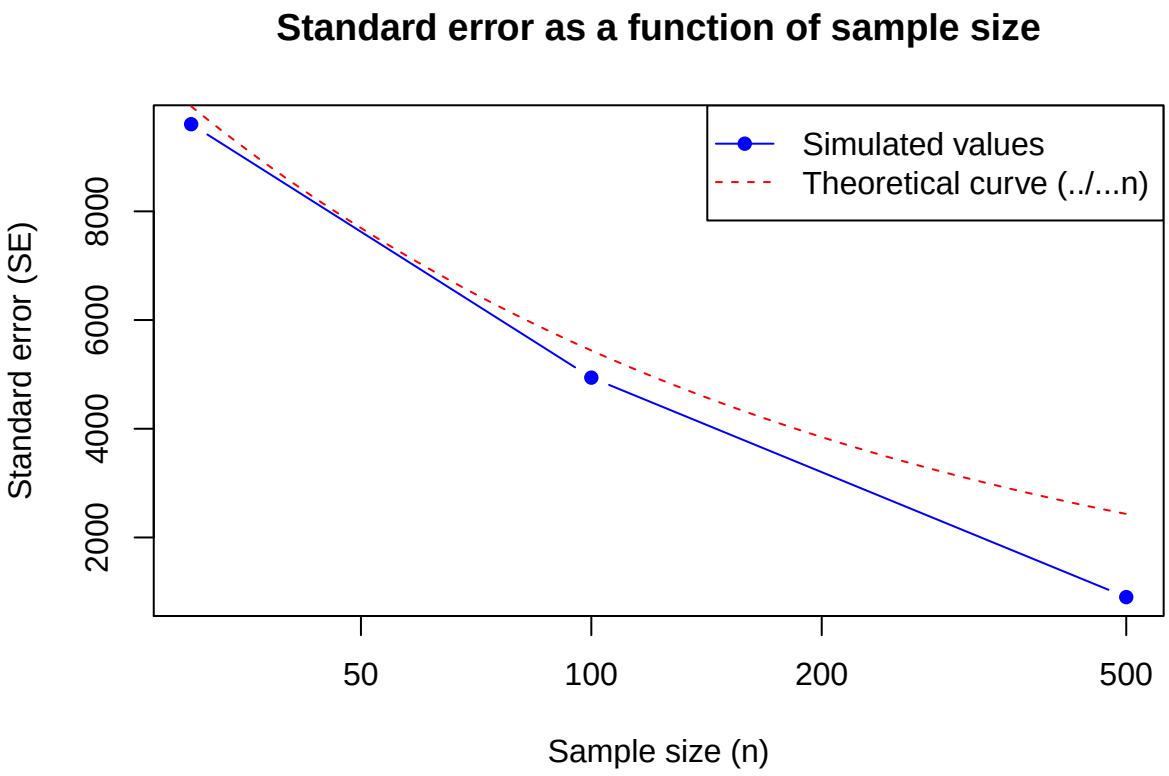
```
## n = 500: Theoretical: 2432.79 Simulated: 903.14
```

```r
# Visualise the relationship between n and SE
n_values <- c(30, 100, 500)
se_values <- c(se_30, se_100, se_500)
plot(n_values, se_values,
     type = "b",
     log = "x",  # Logarithmic x-axis
     main = "Standard error as a function of sample size",
     xlab = "Sample size (n)",
     ylab = "Standard error (SE)",
     col = "blue",
     pch = 16)
# Theoretical curve: SE =  /√n
curve(sigma / sqrt(x), add = TRUE, col = "red", lty = 2, from = 30, to = 500)
legend("topright",
       legend = c("Simulated values", "Theoretical curve ( /√n)"),
       col = c("blue", "red"),
       lty = c(1, 2),
       pch = c(16, NA))
```

```
## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretical curve ( /√n)' in 'mbcsToSbcs': dot
## substituted for <cf>
```

```
## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretical curve ( /√n)' in 'mbcsToSbcs': dot
```

```
## substituted for <83>

## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretical curve ( /√n)' in 'mbcsToSbcs': dot
## substituted for <e2>

## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretical curve ( /√n)' in 'mbcsToSbcs': dot
## substituted for <88>

## Warning in (function (s, units = "user", cex = NULL, font = NULL, vfont = NULL,
## : conversion failure on 'Theoretical curve ( /√n)' in 'mbcsToSbcs': dot
## substituted for <9a>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretical curve
## ( /√n)' in 'mbcsToSbcs': dot substituted for <cf>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretical curve
## ( /√n)' in 'mbcsToSbcs': dot substituted for <83>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretical curve
## ( /√n)' in 'mbcsToSbcs': dot substituted for <e2>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretical curve
## ( /√n)' in 'mbcsToSbcs': dot substituted for <88>

## Warning in text.default(x, y, ...): conversion failure on 'Theoretical curve
## ( /√n)' in 'mbcsToSbcs': dot substituted for <9a>
```



Statistical explanation:

- As the sample size n increases:
  - The distribution of the sample means becomes narrower (smaller spread)

  - The standard error decreases (proportional to $1/\sqrt{n}$)
  - The estimates become more precise
- The relation SE = /√n shows that the standard error decreases with the square root of the sample size.
- This means that to halve the standard error, the sample size has to be quadrupled.
- This diminishing marginal benefit explains why sample sizes between 30 and 1000 are often used in practice.

R explanation:

- The parameter `xlim` in `hist()` sets the limits of the x-axis.
- The function `curve()` draws a function over a given range.
- The parameter `log = "x"` in `plot()` produces a logarithmic x-axis.
- The function `legend()` adds a legend to a plot.

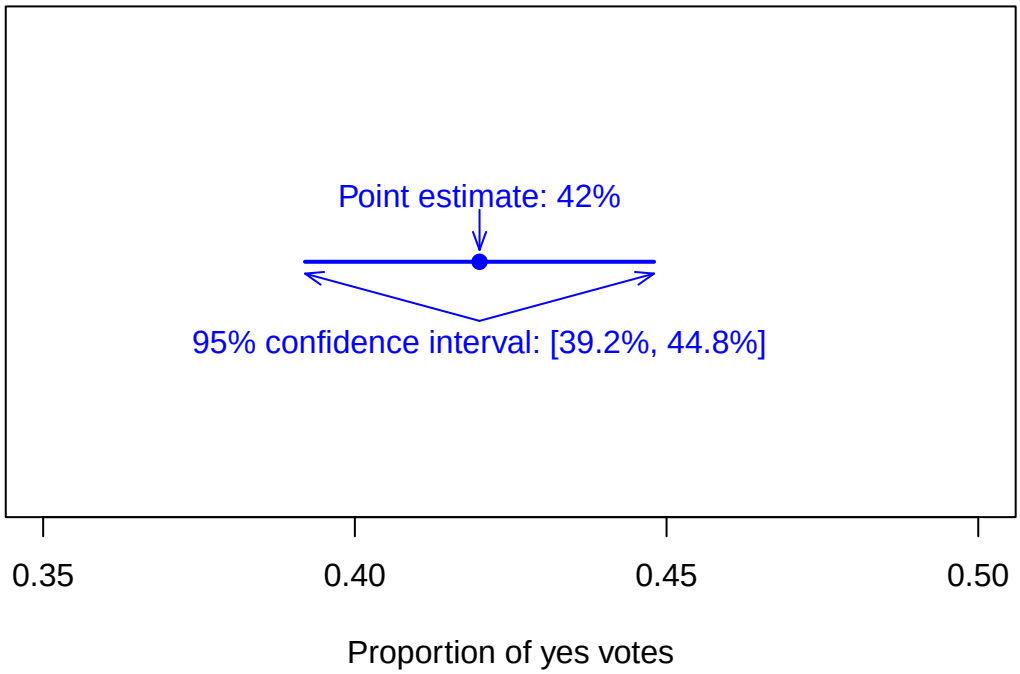
7 Range of Plausible Values

A range of plausible values (often called a confidence interval) indicates the range in which the true population parameter lies with a given probability.

```r
# Example: SRF poll on 9 March 2021
# p̂ = 42% ± 2.8% → plausible values: [0.392, 0.448]
p_hat <- 0.42
error <- 0.028
plausible_interval <- c(p_hat - error, p_hat + error)
cat("Range of plausible values (SRF poll):", round(plausible_interval[1], 3),
    "to", round(plausible_interval[2], 3), "\n")
```

```
## Range of plausible values (SRF poll): 0.392 to 0.448
```

```r
# Visualise the range of plausible values
plot(c(0.35, 0.5), c(0, 0),
     type = "n",
     xlab = "Proportion of yes votes",
     ylab = "",
     main = "Range of plausible values for the proportion of yes votes",
     yaxt = "n")  # No y-axis
points(p_hat, 0, pch = 19, col = "blue")
segments(plausible_interval[1], 0, plausible_interval[2], 0, col = "blue", lwd = 2)
text(p_hat, 0.28, "Point estimate: 42%", col = "blue")
text(mean(plausible_interval), -0.35, "95% confidence interval: [39.2%, 44.8%]", col =
 ↪  "blue")
arrows(p_hat, 0.22, p_hat, 0.05, col = "blue", angle = 20, length = 0.1)
arrows(mean(plausible_interval), -0.25, plausible_interval[1], -0.05, col = "blue", angle
 ↪  = 20, length = 0.1)
arrows(mean(plausible_interval), -0.25, plausible_interval[2], -0.05, col = "blue", angle
 ↪  = 20, length = 0.1)
```



Statistical explanation:

- A **confidence interval** is a range that contains the true population parameter with a given probability (e.g. 95%).
- The general form of a confidence interval is: point estimate ± margin of error.
- The margin of error depends on the desired confidence level, the standard error, and the distribution of the sample statistic.
- For the mean with large n, the margin of error is approximately: z * SE, where z is the z-value for the desired confidence level (e.g. 1.96 for 95%).
- The interpretation of a 95% confidence interval is: if we draw many samples and compute a 95% confidence interval for each, about 95% of these intervals will contain the true population parameter.

R explanation:

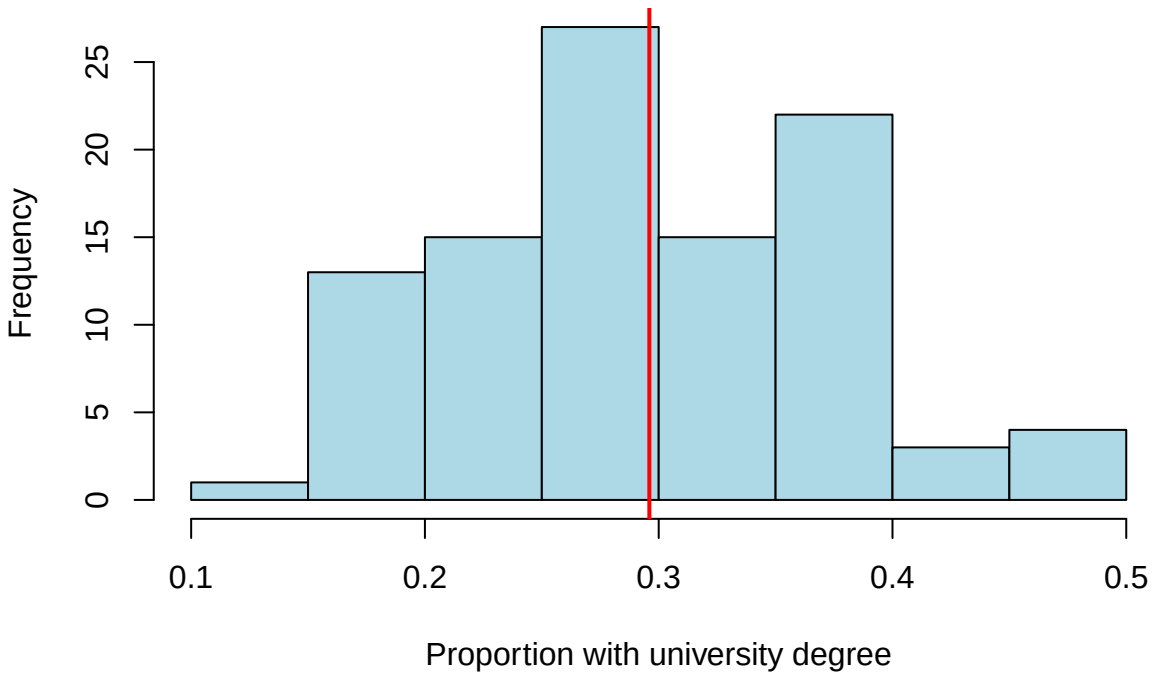
- The function `plot()` with `type = "n"` creates an empty plot.
- The function `points()` adds points to an existing plot.
- The function `segments()` adds line segments to an existing plot.
- The parameter `yaxt = "n"` in `plot()` suppresses the display of the y-axis.

8 Confidence Intervals – Simulation for Proportions

We can investigate the properties of confidence intervals through simulation. Here we simulate the proportion of people with a university degree in Switzerland.

```r
# Example: proportion of university graduates among Swiss women (p = 0.296, i.e. 29.6%)
set.seed(101)
# Draw 100 samples and compute the proportion for each
stich_m <- do(100) * mean(sample(0:1, prob = c(0.704, 0.296), size = 30, replace = TRUE))
```

```r
# Visualise the distribution of the sample proportions
hist(stich_m$mean,
     main = "Sample proportion of university graduates",
     xlab = "Proportion with university degree",
     col = "lightblue")
abline(v = 0.296, col = "red", lwd = 2)
```




```r
# Compute the standard error
se_hat <- sd(stich_m$mean)
cat("Simulated standard error (SE) for the proportion:", round(se_hat, 4), "\n")
```

```
## Simulated standard error (SE) for the proportion: 0.0796
```

```r
# Theoretical standard error for proportions: SE = sqrt(p*(1-p)/n)
p <- 0.296
n <- 30
se_theo <- sqrt(p * (1 - p) / n)
cat("Theoretical standard error (SE) for the proportion:", round(se_theo, 4), "\n")
```

```
## Theoretical standard error (SE) for the proportion: 0.0833
```

Statistical explanation:

- For proportions: the theoretical standard error is SE = √(p*(1-p)/n), where p is the true proportion and n is the sample size.
- The sampling distribution of a proportion is approximately normal for sufficiently large n (the binomial distribution converges to the normal distribution).
- A rule of thumb: the normal approximation is appropriate when np 5 *and* n(1-p) 5.

- In practice, p is often unknown and the standard error has to be estimated by $\sqrt{(\hat{p}*(1-\hat{p})/n)}$, where $\hat{p}$ is the sample proportion.

R explanation:

- The function `sample()` with `replace = TRUE` draws a sample with replacement.
- The parameter `prob` in `sample()` specifies the probabilities of the different values.

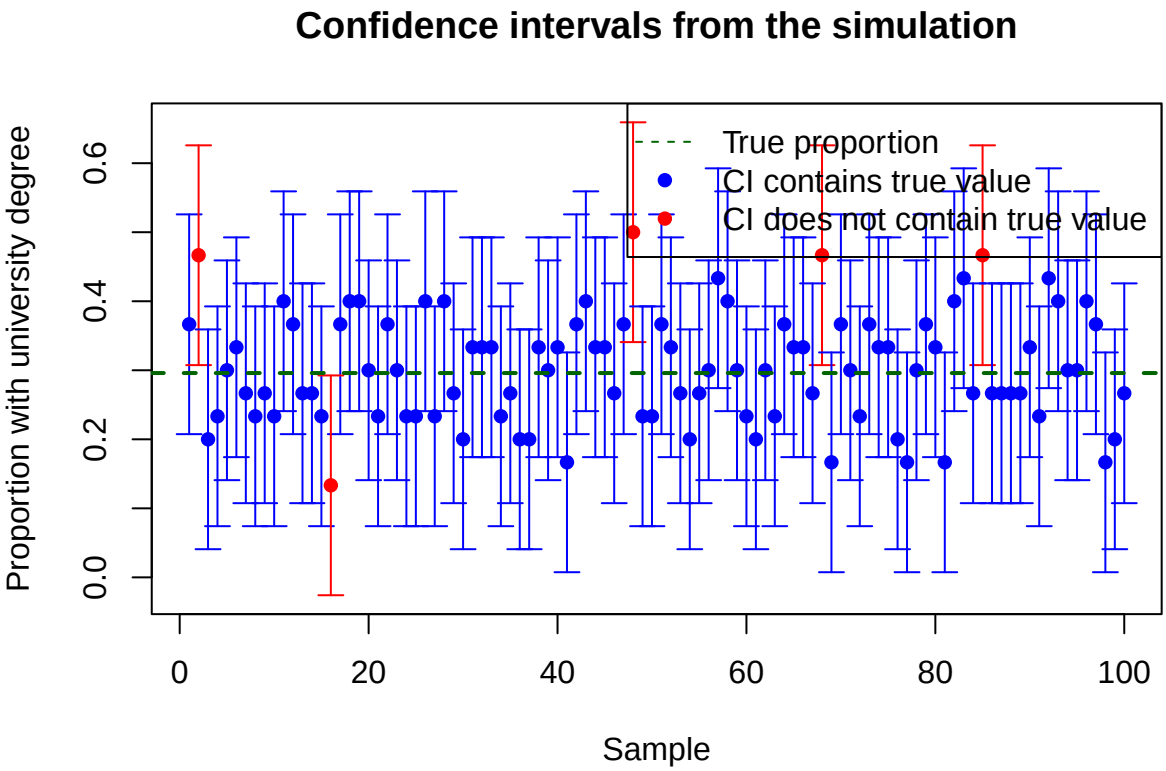
9 Visualising Confidence Intervals from the Simulation

We can compute a confidence interval for each simulated sample and visualise how many of these intervals contain the true population parameter.

```
# For each of the 100 samples compute a 95% confidence interval (x̄ ± 2·SE)
lower <- stich_m$mean - 2 * se_hat
upper <- stich_m$mean + 2 * se_hat

# True proportion
p_true <- 0.296

# Count how many intervals contain the true value
contains_p <- (lower <= p_true) & (p_true <= upper)
share_contains_p <- mean(contains_p)
cat("Proportion of confidence intervals that contain the true value:",
    round(share_contains_p * 100, 1), "%\n")
```

```
## Proportion of confidence intervals that contain the true value: 95 %
```

```
# Visualise the confidence intervals
plotCI(x = 1:100, y = stich_m$mean, li = lower, ui = upper,
       main = "Confidence intervals from the simulation",
       xlab = "Sample", ylab = "Proportion with university degree",
       col = ifelse(contains_p, "blue", "red"),
       pch = 16)
abline(h = p_true, col = "darkgreen", lwd = 2, lty = 2)
legend("topright",
       legend = c("True proportion", "CI contains true value", "CI does not contain true
        ↪  value"),
       col = c("darkgreen", "blue", "red"),
       lty = c(2, NA, NA),
       pch = c(NA, 16, 16))
```



Statistical explanation:

- A **95% confidence interval** means that under repeated sampling about 95% of the computed intervals will contain the true population parameter.
- The width of a confidence interval depends on:
  - The chosen confidence level (higher level → wider interval)
  - The standard error (larger SE → wider interval)
  - The sample size (larger n → narrower interval)
- Confidence intervals convey the precision of an estimate: the narrower the interval, the more precise the estimate.
- The correct interpretation of a single 95% confidence interval is: "We are 95% confident that the computed interval contains the true population parameter."

R explanation:

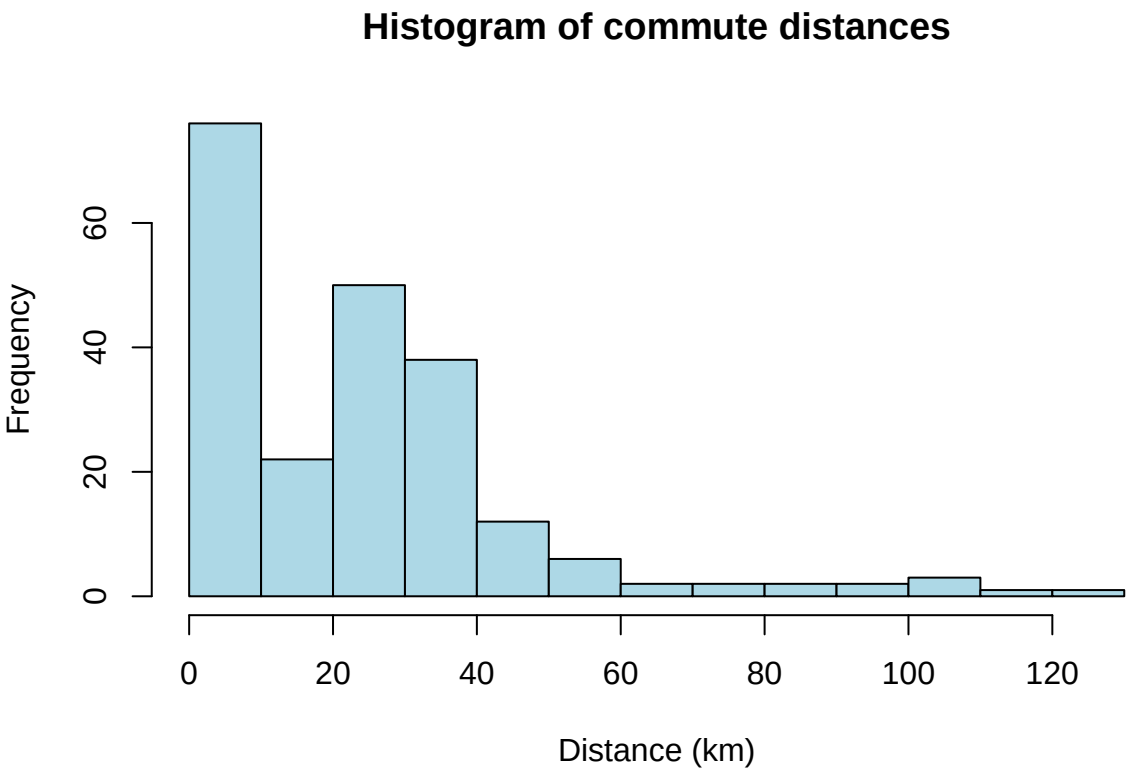
- The function `plotCI()` from the `plotrix` package creates a plot with confidence intervals.
- The parameters `li` and `ui` in `plotCI()` give the lower and upper limit of the confidence intervals respectively.
- The function `ifelse()` returns different values depending on a condition, here for the colour of the points.

10 Bootstrap Samples and Standard Errors

The bootstrap method is a resampling technique in which samples are repeatedly drawn with replacement from the original sample. It allows the estimation of standard errors and confidence intervals without making assumptions about the underlying distribution.

```
# Example: BFH dataset, variable distance (commute distance)
hist(BFH$distance,
```

```
     main = "Histogram of commute distances",
     xlab = "Distance (km)",
     col = "lightblue")
```



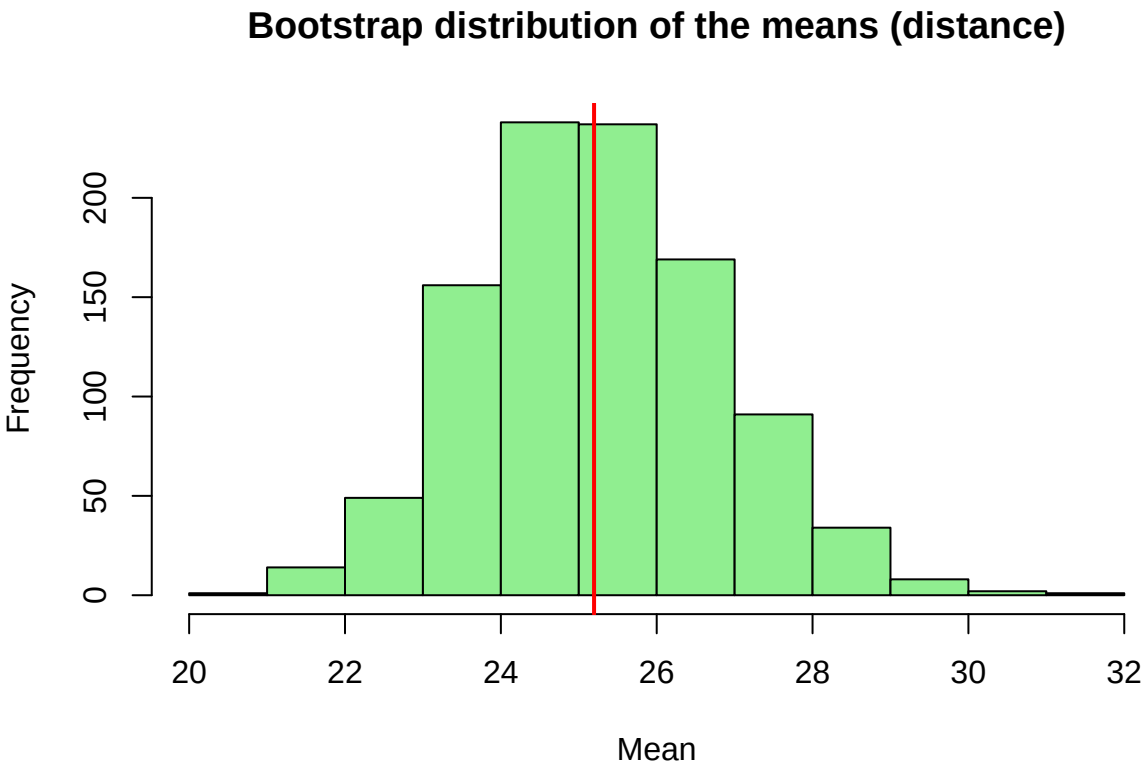

```
# Compute the mean and standard deviation
mean_distance <- mean(BFH$distance)
sd_distance <- sd(BFH$distance)
cat("Mean of commute distances:", round(mean_distance, 2), "km\n")
```

```
## Mean of commute distances: 25.2 km
```

```
cat("Standard deviation of commute distances:", round(sd_distance, 2), "km\n")
```

```
## Standard deviation of commute distances: 22.65 km
```

```
# Draw 1000 bootstrap samples and compute the mean for each
set.seed(202)
boot1000.dist <- do(1000) * resample(BFH$distance)
boot1000.m <- apply(boot1000.dist, 1, mean)

# Visualise the bootstrap distribution
hist(boot1000.m,
     main = "Bootstrap distribution of the means (distance)",
     xlab = "Mean",
     col = "lightgreen")
abline(v = mean_distance, col = "red", lwd = 2)
```




```r
# Compute the bootstrap standard error
dist.se.hat <- sd(boot1000.m)
cat("Bootstrap standard error (distance):", round(dist.se.hat, 2), "km\n")
```

```
## Bootstrap standard error (distance): 1.56 km
```

```r
# Theoretical standard error for comparison
n_bfh <- length(BFH$distance)
se_theo_dist <- sd_distance / sqrt(n_bfh)
cat("Theoretical standard error (distance):", round(se_theo_dist, 2), "km\n")
```

```
## Theoretical standard error (distance): 1.54 km
```

Statistical explanation:

- The **bootstrap method** was developed in 1979 by Bradley Efron and is a computationally intensive method for estimating the sampling distribution of a statistic.
- Core idea: the original sample is treated as a "substitute population" from which samples are repeatedly drawn with replacement.
- For each bootstrap sample, the statistic of interest (e.g. the mean) is computed.
- The distribution of these bootstrap statistics approximates the sampling distribution of the statistic.
- The standard error is estimated as the standard deviation of the bootstrap statistics.
- Advantages of the bootstrap method:
  - No assumptions about the underlying distribution are required
  - Applicable to complex statistics for which no simple formulas exist
  - Works even with small samples

R explanation:

- The function `resample()` from the `mosaic` package draws a sample with replacement from a vector or

data frame.
- The function `apply()` applies a function to the rows (MARGIN = 1) or columns (MARGIN = 2) of a matrix.

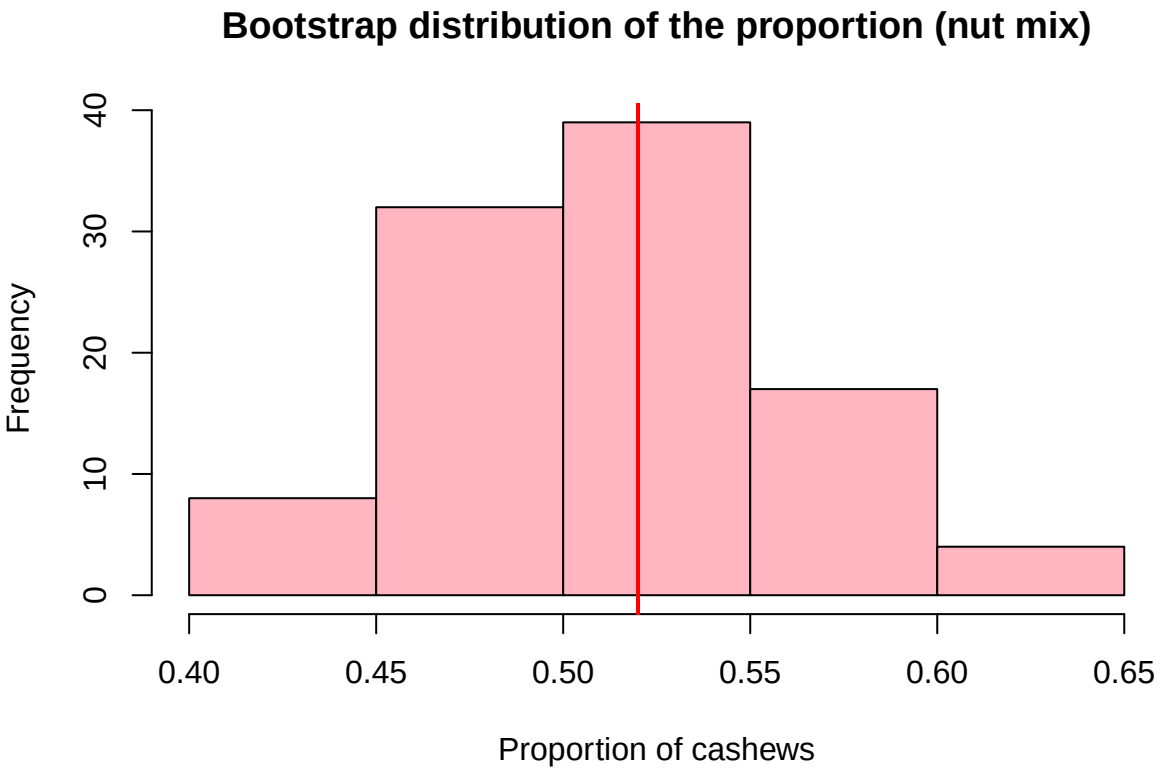
11 Bootstrap: Example – Nut Mix

Another example of the bootstrap method: we estimate the proportion of cashew nuts in a nut mix.

```
# Example: a packet of nut mix contains 100 nuts, 52 of which are cashews.
set.seed(303)
phat <- 0.52
nuts <- sample(c(rep(1, phat * 100), rep(0, (1 - phat) * 100)))
table(nuts)  # Check the frequencies
```

```
## nuts
##  0  1
## 48 52
```

```
# Draw 100 bootstrap samples and compute the proportion for each
nuts.re100 <- do(100) * resample(nuts)
nuts.m100 <- apply(nuts.re100, 1, mean)

# Visualise the bootstrap distribution
hist(nuts.m100,
     main = "Bootstrap distribution of the proportion (nut mix)",
     xlab = "Proportion of cashews",
     col = "lightpink")
abline(v = phat, col = "red", lwd = 2)
```




```r
# Compute the bootstrap standard error
nuts.se.hat <- sd(nuts.m100)
cat("Bootstrap standard error (nut proportion):", round(nuts.se.hat, 3), "\n")
```

```
## Bootstrap standard error (nut proportion): 0.048
```

```r
# Theoretical standard error for proportions for comparison
se_theo_nuts <- sqrt(phat * (1 - phat) / 100)
cat("Theoretical standard error (nut proportion):", round(se_theo_nuts, 3), "\n")
```

```
## Theoretical standard error (nut proportion): 0.05
```

Statistical explanation:

- For estimating proportions, the bootstrap method is particularly useful when the sample size is small or when the assumptions for the normal approximation are not met.
- The bootstrap standard error should be close to the theoretical standard error if the sample size is sufficiently large.
- The bootstrap distribution of a proportion is often not symmetric, especially when the proportion is close to 0 or 1.
- In such cases, percentile-based bootstrap confidence intervals can be more accurate than intervals based on the standard error.

R explanation:

- The function `rep()` repeats a value or vector.
- The function `table()` creates a frequency table.

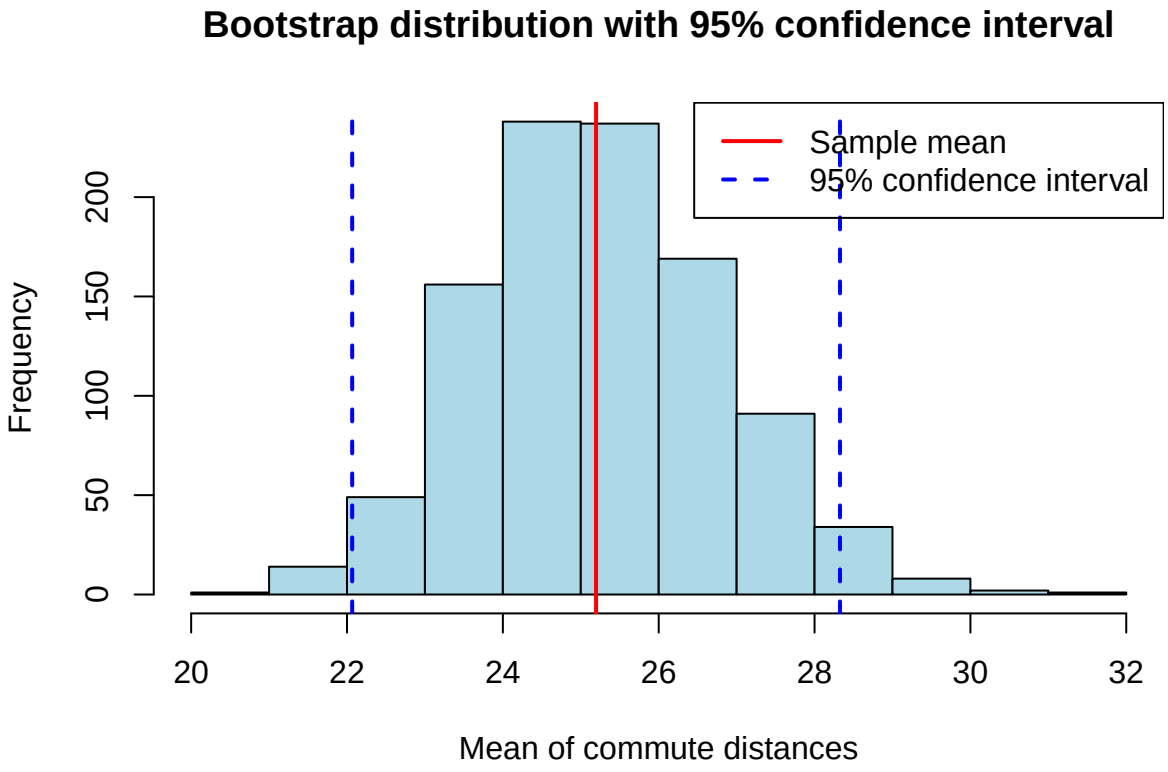
12 Confidence Intervals with the Bootstrap Method (SE Method)

Using the bootstrap standard error we can compute confidence intervals for the mean of the commute distances.

```
# Compute the mean of the commute distances
xbar_distance <- mean(BFH$distance)

# Compute the 95% confidence interval using the SE method
# 95% CI: x̄ ± 1.96*SE (often rounded to x̄ ± 2*SE)
dist.konf <- xbar_distance + c(-1, 1) * 2 * dist.se.hat
cat("95% confidence interval (SE method) for distance:", round(dist.konf[1], 2),
    "to", round(dist.konf[2], 2), "km\n")
```

```
## 95% confidence interval (SE method) for distance: 22.07 to 28.33 km
```

```
# Visualise the confidence interval
hist(boot1000.m,
     main = "Bootstrap distribution with 95% confidence interval",
     xlab = "Mean of commute distances",
     col = "lightblue")
abline(v = xbar_distance, col = "red", lwd = 2)
abline(v = dist.konf, col = "blue", lwd = 2, lty = 2)
legend("topright",
       legend = c("Sample mean", "95% confidence interval"),
       col = c("red", "blue"),
       lty = c(1, 2),
       lwd = 2)
```



Statistical explanation:

- The **SE method** for bootstrap confidence intervals is based on the assumption that the bootstrap distribution is approximately normal.
- The 95% confidence interval is computed as: $\bar{x} \pm 1.96$*SE, where SE is the bootstrap standard error.
- This method is easy to apply and works well when the bootstrap distribution is symmetric.
- For skewed distributions or small samples, the SE method can however be inaccurate.
- In such cases, percentile-based methods are often more appropriate.

R explanation:

- The expression `c(-1, 1)` produces a vector with the values -1 and 1.
- Multiplying by `2 * dist.se.hat` and adding to `xbar_distance` produces the lower and upper limits of the confidence interval.

13 Alternative Confidence Intervals – Bootstrap Percentile Method

An alternative method for computing bootstrap confidence intervals is the percentile method, which uses the percentiles of the bootstrap distribution directly.

```
# Compute the 95% confidence interval using the percentile method
dist.q025 <- quantile(boot1000.m, probs = 0.025, type = 1)
dist.q975 <- quantile(boot1000.m, probs = 0.975, type = 1)
cat("95% confidence interval (percentile method) for distance:", round(dist.q025, 2),
    "to", round(dist.q975, 2), "km\n")
```

```
## 95% confidence interval (percentile method) for distance: 22.29 to 28.43 km
```

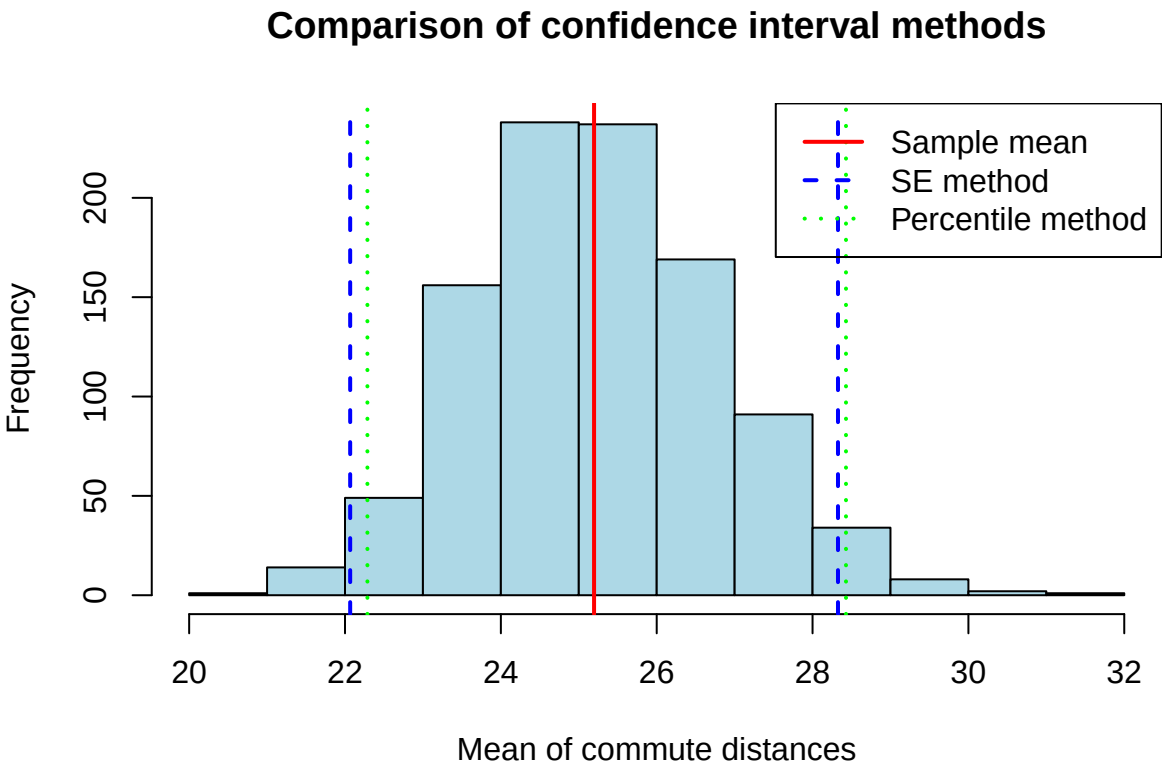
```
# Compare the two methods
cat("SE method:        ", round(dist.konf[1], 2), "to", round(dist.konf[2], 2), "km\n")
```

```
## SE method:         22.07 to 28.33 km
```

```
cat("Percentile method:", round(dist.q025, 2), "to", round(dist.q975, 2), "km\n")
```

```
## Percentile method: 22.29 to 28.43 km
```

```
# Visualise both confidence intervals
hist(boot1000.m,
     main = "Comparison of confidence interval methods",
     xlab = "Mean of commute distances",
     col = "lightblue")
abline(v = xbar_distance, col = "red", lwd = 2)
abline(v = dist.konf, col = "blue", lwd = 2, lty = 2)
abline(v = c(dist.q025, dist.q975), col = "green", lwd = 2, lty = 3)
legend("topright",
       legend = c("Sample mean", "SE method", "Percentile method"),
       col = c("red", "blue", "green"),
       lty = c(1, 2, 3),
       lwd = 2)
```



Statistical explanation:

- The **percentile method** for bootstrap confidence intervals uses the percentiles of the bootstrap distribution directly.
- A 95% confidence interval is defined by the 2.5 and 97.5 percentiles of the bootstrap distribution.
- Advantages of the percentile method:
  - Takes the shape of the bootstrap distribution into account
  - Works for skewed distributions as well
  - Easy to compute and to interpret
- Disadvantages:
  - Can be inaccurate with small samples
  - Does not account for bias in the bootstrap distribution
- For symmetric distributions, the SE method and the percentile method yield similar results.
- For skewed distributions, the results can differ markedly.

R explanation:

- The function `quantile()` computes quantiles (percentiles) of a vector.
- The parameter `probs` specifies the desired percentiles as decimal numbers.
- The parameter `type` specifies the method for computing the quantiles (here: type 1).

14 Confidence Intervals with Percentiles for Different Confidence Levels

We can compute bootstrap confidence intervals for various confidence levels in order to investigate the effect of the confidence level on the width of the interval.

```
# 90% confidence interval
dist.q05 <- quantile(boot1000.m, probs = 0.05, type = 1)
```

```r
dist.q95 <- quantile(boot1000.m, probs = 0.95, type = 1)
cat("90% confidence interval for distance:", round(dist.q05, 2), "to", round(dist.q95,
 ↪  2), "km\n")
```

```
## 90% confidence interval for distance: 22.81 to 27.92 km
```

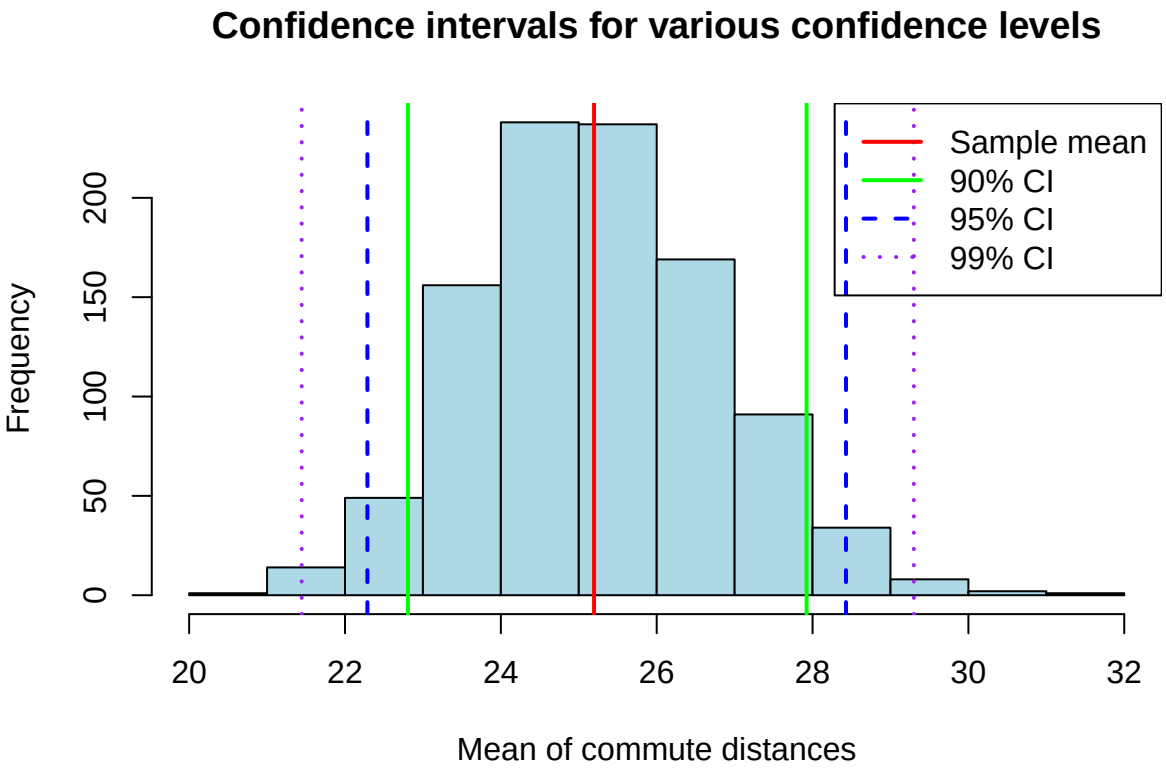
```r
# 95% confidence interval (already computed)
cat("95% confidence interval for distance:", round(dist.q025, 2), "to", round(dist.q975,
 ↪  2), "km\n")
```

```
## 95% confidence interval for distance: 22.29 to 28.43 km
```

```r
# 99% confidence interval
dist.q005 <- quantile(boot1000.m, probs = 0.005, type = 1)
dist.q995 <- quantile(boot1000.m, probs = 0.995, type = 1)
cat("99% confidence interval for distance:", round(dist.q005, 2), "to", round(dist.q995,
 ↪  2), "km\n")
```

```
## 99% confidence interval for distance: 21.44 to 29.3 km
```

```r
# Visualise the various confidence intervals
hist(boot1000.m,
     main = "Confidence intervals for various confidence levels",
     xlab = "Mean of commute distances",
     col = "lightblue")
abline(v = xbar_distance, col = "red", lwd = 2)
abline(v = c(dist.q05, dist.q95), col = "green", lwd = 2, lty = 1)
abline(v = c(dist.q025, dist.q975), col = "blue", lwd = 2, lty = 2)
abline(v = c(dist.q005, dist.q995), col = "purple", lwd = 2, lty = 3)
legend("topright",
       legend = c("Sample mean", "90% CI", "95% CI", "99% CI"),
       col = c("red", "green", "blue", "purple"),
       lty = c(1, 1, 2, 3),
       lwd = 2)
```



Statistical explanation:

- The **confidence level** is the probability with which the computed interval contains the true population parameter.
- Higher confidence levels yield wider intervals:
  - 90% CI: uses the 5th and 95th percentile
  - 95% CI: uses the 2.5th and 97.5th percentile
  - 99% CI: uses the 0.5th and 99.5th percentile
- The choice of confidence level depends on the application context:
  - Higher levels (e.g. 99%) provide more certainty but less precision
  - Lower levels (e.g. 90%) provide more precision but less certainty
- In practice, a confidence level of 95% is often used, as it represents a good compromise between certainty and precision.

R explanation:

- The various line types (`lty`) help to visually distinguish the different confidence intervals.

15 Confidence Intervals for Differences

We can also use the bootstrap method to compute confidence intervals for differences between groups, e.g. for the difference in average exercise times between men and women.

```r
# Example: difference in average exercise hours between men and women
men <- ExerciseHours[ExerciseHours$Sex == "M", ]
women <- ExerciseHours[ExerciseHours$Sex == "F", ]

# Compute the means and the difference
mw.m <- mean(men$Exercise)
mw.f <- mean(women$Exercise)
```

```r
diff.hat <- mw.m - mw.f
cat("Mean men:", round(mw.m, 2), "hours\n")
```

```
## Mean men: 12.4 hours
```

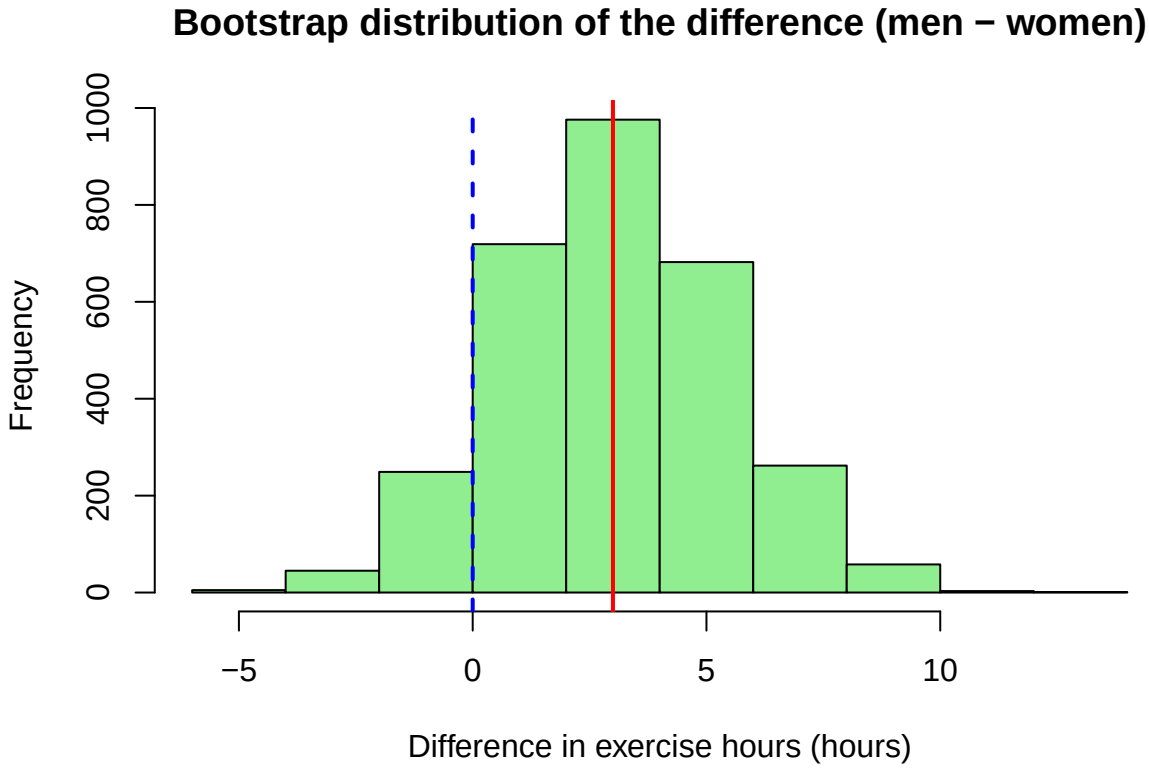
```r
cat("Mean women:", round(mw.f, 2), "hours\n")
```

```
## Mean women: 9.4 hours
```

```r
cat("Difference (men - women):", round(diff.hat, 2), "hours\n")
```

```
## Difference (men - women): 3 hours
```

```r
# Bootstrap for differences: draw 3000 bootstrap samples
set.seed(404)
boot3000.diff <- do(3000) * (mean(resample(men$Exercise)) -
 ↪  mean(resample(women$Exercise)))

# Visualise the bootstrap distribution of the difference
hist(boot3000.diff$result,
     main = "Bootstrap distribution of the difference (men - women)",
     xlab = "Difference in exercise hours (hours)",
     col = "lightgreen")
abline(v = diff.hat, col = "red", lwd = 2)
abline(v = 0, col = "blue", lwd = 2, lty = 2)
```



```r
# Compute the 95% confidence interval using the percentile method
diff.q025 <- quantile(boot3000.diff$result, probs = 0.025, type = 1)
diff.q975 <- quantile(boot3000.diff$result, probs = 0.975, type = 1)
cat("95% confidence interval for the difference:", round(diff.q025, 2), "to",
 ↪  round(diff.q975, 2), "hours\n")
```

```
## 95% confidence interval for the difference: -1.58 to 7.75 hours
```

```r
# Check whether the confidence interval contains 0
contains_zero <- (diff.q025 <= 0) & (0 <= diff.q975)
cat("Does the confidence interval contain 0?", ifelse(contains_zero, "Yes", "No"), "\n")
```

```
## Does the confidence interval contain 0? Yes
```

```r
cat("Interpretation: the difference is", ifelse(contains_zero, "not", ""), "statistically
 ↪  significant.\n")
```

```
## Interpretation: the difference is not statistically significant.
```

Statistical explanation:

- Confidence intervals for **differences** are particularly useful for assessing differences between groups.
- The bootstrap method for differences involves the following steps:
  1. Draw bootstrap samples from both groups
  2. For each pair of bootstrap samples, compute the difference of the means
  3. Use the distribution of these differences to compute a confidence interval
- An important aspect for the interpretation: does the confidence interval contain 0?
  - If yes: the difference is not statistically significant (at the corresponding level)
  - If no: the difference is statistically significant
- This method is equivalent to a hypothesis test for the difference between the groups.

R explanation:

- The expression `ExerciseHours[ExerciseHours$Sex == "M", ]` filters the rows of the data frame for which the column `Sex` has the value "M".
- The function `ifelse()` returns different values depending on a condition, here for the interpretation of the confidence interval.

16 Confidence Intervals for Correlations

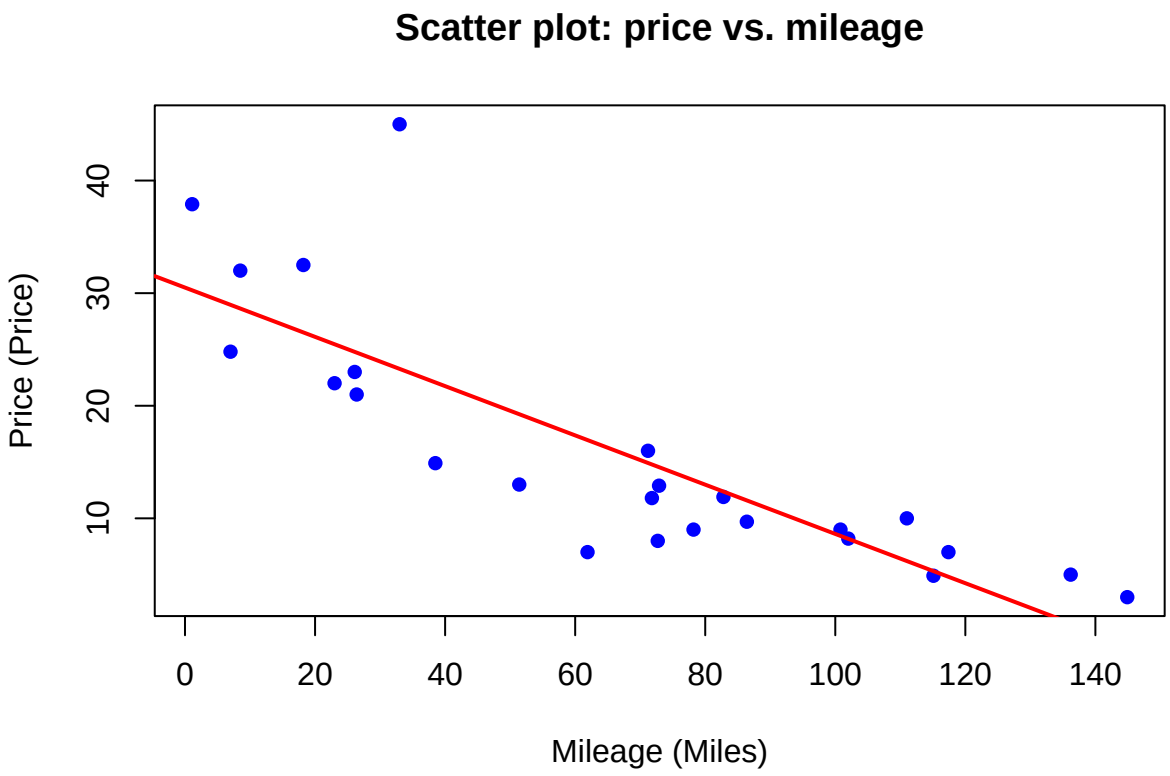
Finally, we can also use the bootstrap method to compute confidence intervals for correlations, e.g. for the correlation between price and mileage of Mustang cars.

```r
# Example: determine the correlation between Price and Miles in the Mustangs dataset
cor_mustangs <- cor(Mustangs$Price, Mustangs$Miles)
cat("Correlation between Price and Miles:", round(cor_mustangs, 3), "\n")
```

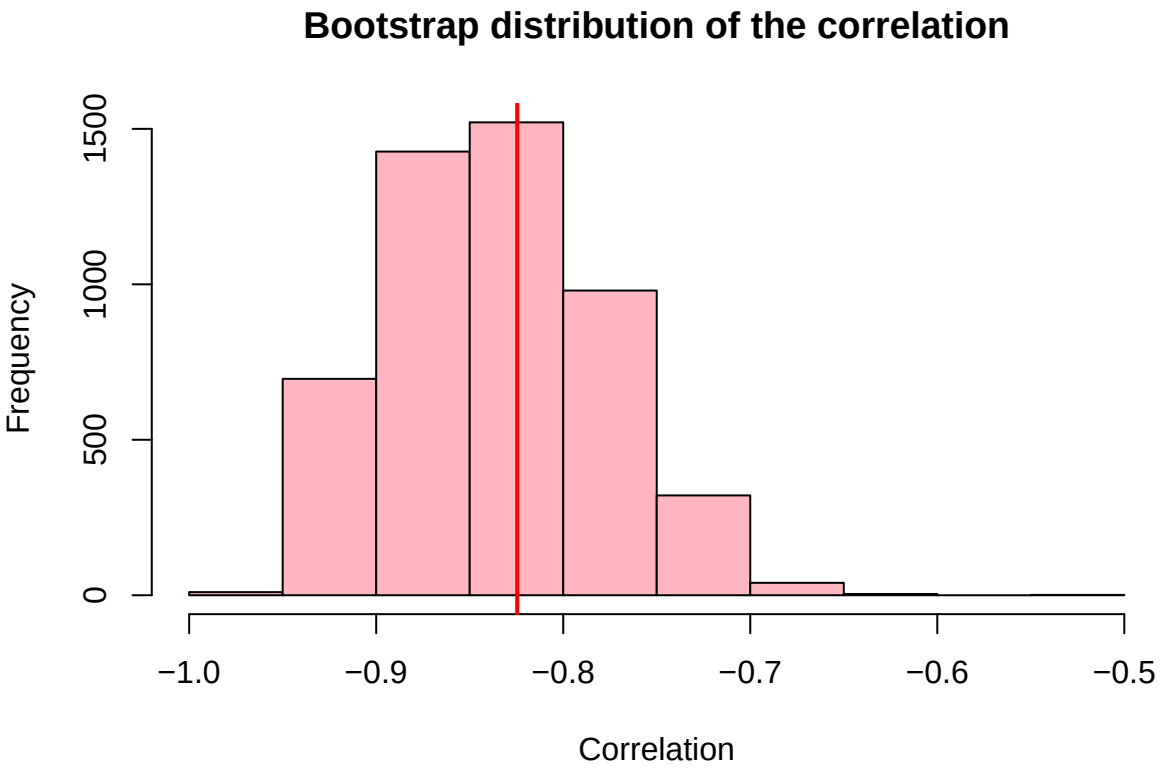
```
## Correlation between Price and Miles: -0.825
```

```r
# Scatter plot
plot(Mustangs$Miles, Mustangs$Price,
     main = "Scatter plot: price vs. mileage",
     xlab = "Mileage (Miles)",
```

```
     ylab = "Price (Price)",
     pch = 16,
     col = "blue")
abline(lm(Price ~ Miles, data = Mustangs), col = "red", lwd = 2)
```




```
# Bootstrap for correlations: draw 5000 bootstrap samples
set.seed(505)
mustangs.cor.boot <- do(5000) * cor(Price ~ Miles, data = resample(Mustangs))

# Visualise the bootstrap distribution of the correlation
hist(mustangs.cor.boot$cor,
     main = "Bootstrap distribution of the correlation",
     xlab = "Correlation",
     col = "lightpink")
abline(v = cor_mustangs, col = "red", lwd = 2)
```




```r
# Compute the 98% confidence interval using the percentile method
cor.q01 <- quantile(mustangs.cor.boot$cor, probs = 0.01, type = 1)
cor.q99 <- quantile(mustangs.cor.boot$cor, probs = 0.99, type = 1)
cat("98% confidence interval for the correlation:", round(cor.q01, 3), "to",
 ↪  round(cor.q99, 3), "\n")
```

```
## 98% confidence interval for the correlation: -0.937 to -0.703
```

```r
# Alternative using the qdata function from the mosaic package
quantiles <- qdata(~ cor, c(0.01, 0.99), data = mustangs.cor.boot)
cat("98% confidence interval (using qdata):\n")
```

```
## 98% confidence interval (using qdata):
```

```r
print(quantiles)
```

```
##         1%        99%
## -0.9369749 -0.7025379
```

```r
# Check whether the confidence interval contains 0
contains_zero <- (cor.q01 <= 0) & (0 <= cor.q99)
cat("Does the confidence interval contain 0?", ifelse(contains_zero, "Yes", "No"), "\n")
```

```
## Does the confidence interval contain 0? No
```

```r
cat("Interpretation: the correlation is", ifelse(contains_zero, "not", ""),
 ↪  "statistically significant.\n")
```

```
## Interpretation: the correlation is  statistically significant.
```

Statistical explanation:

- The **correlation** is a measure of the linear relationship between two variables and lies between -1 and +1.
- The bootstrap method for correlations involves the following steps:
  1. Draw bootstrap samples from the dataset (pairs of values are kept together)
  2. For each bootstrap sample, compute the correlation
  3. Use the distribution of these correlations to compute a confidence interval
- The interpretation of the confidence interval is similar to that for differences:
  - Does the interval contain 0? If not, the correlation is statistically significant
  - The width of the interval indicates the precision of the estimate
- The bootstrap method is particularly useful for correlations, as the sampling distribution of the correlation is often not normal, especially with small samples or extreme correlations.

R explanation:

- The formula `Price ~ Miles` in `cor()` indicates that the correlation between the variables `Price` and `Miles` should be computed.
- The function `qdata()` from the `mosaic` package computes quantiles for a variable in a data frame.
- The function `lm()` fits a linear model to the data, here for the regression line in the scatter plot.

17 Summary

In this lecture we have got to know fundamental concepts of inferential statistics:

1. **Sampling variation**: different samples from the same population yield different estimates.
2. **Standard error**: a measure of the precision of a sample statistic, which decreases as the sample size grows.
3. **Bootstrap method**: a resampling technique for estimating standard errors and confidence intervals without making assumptions about the underlying distribution.
4. **Confidence intervals**: ranges that contain the true population parameter with a given probability.

These concepts are fundamental for statistical inference and allow us to draw conclusions about a population from samples and to quantify the uncertainty of our estimates.

The bootstrap method is particularly useful because it:

- Does not require any assumptions about the underlying distribution
- Is applicable to a wide variety of statistics
- Works even with small samples
- Is easy to implement